

Active Inference LiveStream 058.1 ~ "From pixels to planning: scale-free a

Livestream | 1:58:21

Hello and welcome. This is Active Inference live stream 58.1 on November 7th, 2024.

Welcome to the ActInf Institute. We're a participatory online institute that is communicating, learning, and practicing applied active inference. You can find us at links here. This is a recorded and archived live stream, so please provide feedback so we can improve our work. All backgrounds and perspectives are welcome and will follow video etiquette for live streams. Head over to activeinference.institute to learn more about live streams and other projects. We are in the second in the 58 series. Arun and I last week did 58.0 with some background and context. Kind of the approach to scratch the surface. And today we're going to pick it up with Lance and looking forward to this discussion. So people watching live, please write any live chats. Arun has also prepared a bunch of questions. Let's begin though with Lance if you want to say hi and welcome back. And then any overviews or any introductions, what's the question that you feel will help set the table for today? Thank you.

Well, thank you so much. And thank you so much for having me. So yeah, I'm super excited about this discussion. And I feel a lot of us are trying to understand renalizing objective models. So I'm sure I'm going to learn a lot as well. Yeah. So just to give a little bit of background. So now Kyle Friston is the chief scientific officer at Versus. And one of the challenges that we are kind of like moving towards, and this is public information, is of course scaling active inference. And there's one particular challenge that we're looking at. It's Atari. So for those who don't know, Atari is just a suite of like about 30, 31 games. They've been around for a long time, maybe since the 80s, or I don't know exactly. It's a famous benchmark for reinforcement learning. Now there's been a lot of work in RL solving Atari and maybe stuff even like 10 years old already that solves Atari. So the question is, why are we trying to do that with active inference? And the answer is, all of these RL methods, they use a lot of data. They're super data hungry, and they're not, so they don't have this human level efficiency that we're looking for. And so that's where active inference could potentially come into play. So one of the things that we're looking at at the company is scaling active inference to show that we can solve Atari with a human level efficiency. And so this scale-free approach, planning from pixels, emerged out of these efforts. And this has been work led by Carl. So he's, you know, come up with the ideas and the code and so forth. And, and yeah, and so now we're still pushing with this approach and other approaches. So yeah, it's super exciting. And there's many, many research questions of things we could explore. So very excited to discuss it today.

Okay. Do you want to give any connective tissue between some of your previous conversations or papers or streams? And then we'll jump into the listed questions we have.

Sure. So, so for those who don't know me, so I'm Lance, I've been working on the principle active inference for a long time, primarily on the, on the mathematical side of things. So

back when I started, possibly the only person understanding active inference was Carl himself. And if he privileged people around Carl. And so a lot of the work I did was to try to understand things and put them on a more rigorous mathematical footing. And so we've had a, a few conversations here, a few live streams, speaking about some of that. So we'd say the, the RGM work that we're discussing today is a, is a bit different from, from what I've been working so far. So, previous work was mostly on the theory of the principle theory of active inference, the math. Today, it's more about a piece of code that does things, and it does really cool things. And so it's a much more pragmatic agenda. And I'm personally moving in this more pragmatic set of things, but I have a lot to learn. Yeah. So this is, this is how things stand at the moment. Yes, it was live stream 52 on the geometric methods for sampling optimization, inference and adaptive agents. That was a great and very epistemic setting. And then also now that I'm seeing it also 26. So it was sort of a 26 52 full card Lance situation, but both of these were some of the clearest and most fundamental papers and unpackings in the series.

Arun, anything you want to begin with, or we can just jump into the questions you wrote.

Uh, I mean, first of all, just to say like, I need to catch up on those because they sound great.

Um, also, I think some of my questions are still fairly, like, I don't want to say like towards the end of the paper, they're much more like discussion points, but I don't know if maybe for some people who might not been able to catch the first session to maybe go into a very high level summary of the paper first, uh, and then set some background that might make the discussion points, uh, a bit clearer as they happen. Uh, but very happy to, for you to take the lead on that one, Daniel.

Daniel

Yeah. Lance, what would, what would you say is the overall summary through line and then we'll turn to the specific questions.

Sure. Um, so, so I'll give a quick summary.

Yeah. Um, right. So, um, yeah, so, so the problem that we're looking at is planning from pixels.

So you have a bunch of pixels and we want to, you know, the decision making.

Um, so for, for those who are familiar with active inference, there's typically two flavors of active inference. There's the reflexive type of active inference where you're not really planning in the future, but you're like, uh, if, if I infer it and I'm in this state, then I should do this.

And if I infer, I'm in this state, I should do that.

And it's typically what we see and continuous active inference.

Um, and then there's the other favor of active inference and also sophisticated inference, which is typically in the speech states.

And, um, and that has planning that has expected free energy. It has expiration and so forth.

Um, here we are in a setting when, where we are doing things in the speech states, but we are reflexive.

So this is just to situate with the other things that are in the literature.

And so what happens is we, we get a bunch of pixels.

So yeah, just to go back a little bit, this is, um, the pixels of the planning paper is a way to

learn from expert data.

So if I, if I look at expert data of an agent doing something, I'm going to be able to learn that with an RGM and reproduce it in some kind of a, like a habitual reflexive loop, but I'm not going to do any kind of the, no, for learning.

Um, and, and yeah, and so then there's like a very specific, um, architecture for the RGM generative model, uh, that allows it to do what it, what it does.

And there's a way we can, you know, learn these things, uh, but that's kind of like the, yeah, goal idea or, or a little context.

Okay.

Awesome.

Yeah.

Just, we have the figures if there's any to jump to, let's go to the first, uh, question.

So how are renormalizability and sparsity linked?

And also if you could connect that to how else we've seen sparsity discussed in the Markov blanket concept, what's sparsity in a graphical model?

How is renormalizability doing something similar or different than the sparsity conditions that we've studied before?

Right.

Um, so, um, yeah, just to give a bit of, uh, a bit of context on that question, because you talked about Markov blankets.

So, uh, Cal and a number of people have this idea that living systems and I guess systems in general are made of Markov blankets, I mean, they're themselves Markov blankets.

And so if you take an organism, it would be composed, for example, of, uh, organs with their own Markov blankets and themselves, uh, made of cells with their own Markov blankets.

Uh, and, and you can, you know, subdivide, uh, many, many times sort of zoom in and you'll still see Markov blankets everywhere.

And so the, what, what you get from that, or, or probably what Cal, uh, got from that is that there is some kind of, uh, scale free invariance about the nature of things.

Um, which is that if you, if you zoom in or if you zoom out and, um, you know, arbitrary zooming or zooming, you would still have this kind of like fundamental structure.

And, um, and I should say, uh, the renormalization group is a mathematical apparatus for zooming in or zooming out on things.

And, and so basically these, uh, renormalizing, you can't, and models, they have this, um, assumption that the world is renormalizable, that the world has some kind of a scale free information.

Uh, within them.

And, uh, and so we can get more into the details of that, but basically long story short is that this renormalization aspect, it gives you a very powerful inductive bias on the structure of the generative model.

Um, and in this case, it gives you this kind of renormalizing genital model, which is a genital model that looks the same at every scale.

So it's a genital model that if you, uh, you know, give it pixel data that sort of very zoomed in, uh, that part of the model that feeds that data is going to be the same in terms of its structure than the high level parts of the model that look at like much more causal grain features.

Um, and, and these models are very sparse.

Um, yeah, just because, uh, when you look at the representations in the model, there's many, many, uh, connections are missing, but by design on purpose, right?

So, to have a lot of, uh, sparsity, uh, so it means that they're very lightweight.

Um, and yeah, I would say that's kind of, um, the main advantage, right?

Uh, so when we, when we compare with, uh, traditional machine learning architectures, this is not to say that, you know, this approach has a lot of limitations, but one, one of the strengths is that we, we get some very lightweight models that can learn things with a relatively low number of states.

And, and I think that goes back to sparsity.

Okay.

Let's look at the figure and, and talk about the architecture.

Maybe continue a little bit on what, what specifically

can, can, uh, where, how can we see that renormalization structure in figure three?

And how does this have similarity and differences than like the textbook generative models that we've worked with?

Right.

So, so I should say a full disclaimer, I don't think the explanation of, uh, renormalizing data models is very clear in the paper.

And some will say it's extremely unclear.

Uh, so I'm going to try to, to clear some of that confusion.

Uh, but there's also a part of it where, where I'm probably still confused.

So, so apologies in advance if I cannot clear all the confusion.

Um, but so, so I can try to describe the, the figure that you have here.

And then I, I can also give you my own, uh, kind of personal summary of gender models, uh, of renormalizing gender models.

So, so here we have kind of, um, a classic, a classic, uh, hierarchical partially observed, uh, market decision process.

And the special features is that, um, there's a time horizon of one at each layer.

Um, and then you, you have this renormalizing scale-free aspect, where, as you can see at the bottom level, you have two states, two actions.

These are grouped into one state, one action of the level above.

And then if you go one level, even, even further above, there's, again, two states at the level below and two actions that are grouped into one state and one action the level above.

And you can go like that for as many layers as you want.

Um, in, in the current code, what you do is you specify the number of layers that, that you want, uh, to start with.

And then you would kind of learn, uh, you know, the transition mappings and, and so on and so forth.

What aspects of the code are available or what languages or timeline for overall applying it?

Um, you mean for, for applying it in, you know, to machine learning test?

Is it, is it, is it, or, or just for any of these settings, what is the code availability

or, or what do you think are the pro, the promising ways today and going forward for

people who are interested in the open source active inference implementations to work with it?

Right.

Um, yeah.

So, so I'm not entirely sure, um, as to the kind of like privacy constraints on the, on the code right now.

Um, I would assume there's a link in the paper pointing to a repo, but if that's not the case, it's not the case.

Okay.

Well, um, so I know it's on a private version of SPM.

Um, and so if it hasn't been released yet, uh, then there surely, uh, talks inside the company of like, when to release it, how to release it.

Um, yeah.

So I'm not exactly sure about the timeline of that, unfortunately.

Okay.

Okay.

All good.

Yeah.

I mean, the, it's, it's an interesting, that, uh, piece with the research and being able to share that.

And with this information here from the pseudo code, it, it gives a lot of the recipe and the anatomy of the, uh, model.

But let me just, uh, is there, is there another part of this figure or another figure to continue specifying a little bit more about what's happening or do we want to jump back to the questions?

Um, yeah.

So, uh, there's a presentation by, by Tim Ferberlin that, that, um, um, and there's a useful slide on, on that presentation, uh, that, that could potentially show.

I don't know if I can share my screen here, but I think it would be quite useful to illustrate a bit more.

Sure.

What's going on.

Oh yeah.

Share your screen.

And, uh,

I said, see if I can do that.

This is a new computer.

Um,

Okay.

Yeah.

Unfortunately I'm not going to be able to do it because it's, um,

I mean, I could, but I need to fiddle with the settings or you could, you could email it and I don't have admin rates.

Okay.

You could email any slides.

Um, if you wanted to,

to, uh, admin.

Yeah.

I'll check, I'll check and, um, yeah, I'll get back to you on that.

Okay.

But I can, I'm happy to explain.

Yeah.

Continue.

Um, what, what I was going to show.

So the idea, and, and here I'm going to have to wave my hands a bit, but hopefully it's, uh, it's clear.

I'm going to show you the picture.

Um, you get some pixel data, you get a movie, but basically that's what our RGMs do.

You get a movie of an agent doing something and you're going to learn a gentle model that, you know, learns that and is able to reproduce the agent in the movie.

And so it's able to learn its actions and reproduce its actions depending on the state.

So the way it works is you, um, you take one frame of the movie and you take the next frame.

And, and this would be like your low level observation for the renormalizing during the model.

So why do we take two consecutive frames, uh, as one observation, because we want to extract position, but also momentum, uh, information.

So, so that's our observation to consecutive frames.

Then each frame we're gonna, um, we're gonna divide it into several overlapping patches.

So this, like, uh, convolutional neural network would, so, um, yeah, you have like multiple patches that overlap each center, for example, around pixel.

And then each of those patches, you do a singular value decomposition, which is that you, uh, you take the eigenvectors.

So you have those pixel vectors, you take the eigenvectors and, and you essentially do a PCA.

And, and the reason why you do that, so, so just to give a little bit of, uh, context.

Um, well, first of all, uh, planning is, is much easier in discrete states.

Uh, so somehow things work a lot better, or, or, or at least that's what we think, uh, at this moment.

Uh, so we want to convert this kind of like very high dimensional, kind of continuous data, that are pixels into some more, uh, manageable size, uh, low dimensional discrete representation that we can then work with in the RGM.

And so there's potentially many ways you can do that.

And one way would be to use, uh, like kind of, uh, for example, um, uh, variational autoencoder, where you would then discretize the latent space.

That would be one way to get from continuously discrete.

So there's many, many ways, uh, you can do this step.

The reason why, uh, I suppose, Carl chose SVD is because it's very lightweight.

And it's also very, um, I would say context sensitive.

If you gave one bit of data, I will adapt to that data.

If you give it another bit of data, I will adapt as well.

Um, there's arguably also pros to using a deep learning architecture.

The cause is that it would be need to be trained.

So lots to explore there.

But for now we use it as GD.

And so, um, so once you've done your, your singular value decomposition, you get these eigenvectors that, um, you still get a list of eigenvectors for, for each patch that tell you, okay, well, this is like the most important feature in, in the patch of pixels.

This is second most important feature, third most important feature and so forth.

What you do is you truncate that.

So you take the most important features and then you discretize that.

Um, so it's, it's a lot of stats, um, I know.

So hopefully I can, um, send you a picture.

But the point is that you extract the features using SVD.

So you get a much lower dimensional, uh, representation of the data.

And because you truncate it, you kind of eliminate the noise, you eliminate the things you don't care.

And then this low dimensional thing, which is still continues, because these are eigenvectors, you can discretize it using multiple bins.

So now you have discrete data.

And once you have your discrete data, then you can fit it to the picture that you have in the slide right there and do your hierarchy point.

And kind of like much more classic.

Yeah.

Awesome.

Yeah.

And the dot zero, we talked a little bit about that process about going from the continuous to the discrete to enable planning over the discretized form.

Arun, go for it.

Yeah.

And thanks for that summary, Lars.

That was really like clear and informative.

Uh, one question I have just to follow off on that is when you said something you said quite early on, which is that planning in the continuous space is really difficult.

Um, could you say a little bit about why that is?

So the reason I ask is, I think it's a very common thing that people would want to do is have an agent that can plan an action to do this thing, but control the degree at which it does it.

So I would like to move, but by a certain amount, how do I choose how much to move in a continuous distance?

So I guess maybe could you say a little bit about why it's so difficult to plan in continuous space?

Yeah, it's a really good question. So I think, um, so planning is always about doing things.

Um, so when, when I talk about planning, I don't talk specifically about action selection.

I talk about imagining what will happen, um, you know, in a certain time interval in the future.

So it means that you're looking at, uh, what would happen if I did that in terms of what is the trajectory of what would happen if I did that.

Now, if you're in discrete spaces with time, you're looking at a sequence of future states.

But if you're in, um, and, and so your space of trajectory is that you're kind of like planning over is discrete.

Because it's like, um, possible steps of the next time step and then 10 possible step of the next time step.

So it can be huge, but it's discrete.

When you're in continuous space, if you want to plan, uh, in a certain time and to run the future, you're looking at trajectories of what would happen if you took a particular trajectory of actions.

And so if you're in continuous space, continuous time, the space of trajectories is infinite dimensional.

So I think that's kind of like the, well, it's infinite and it's also infinite dimensional.

So it's kind of like really bad.

And yeah, and there's probably, I mean, there are algorithms that kind of do this.

I'm not really an expert on that, but I think that's kind of like the underlying limitation, the, the fundamental, fundamental mathematical limitation.

And so the thing you can do then, uh, to plan in continuous space, continuous time is you abstract

away on a discrete space and then, and then you go back down and, and this is what, what we've been doing.

I mean, as you know, um, in active inference, it's kind of like the, the hack, uh, there's probably other ways to do it, but yeah, that's where we're at.

So maybe something just to clarify there is the nature of these infinities, right?

So you said infinite dimensional, but then also it's almost like infinite values for each of those dimensions.

So that's why it gets really explosive in the number of things you have to consider.

Uh, one of the things that we talked about in the dot zero was the path-based formulation and how that seemed to be even more combinations you would have to consider compared to the textbook, uh, formulation where now we're considering paths over states and actions. And there are as many, uh, combinations of paths of like common, combine, combining states and action choices, uh, which already sounded like quite difficult to solve relative to sort of the toy examples of just what, what's in the book.

So maybe you could, uh, say a little bit about that.

Right.

So, so just to make sure I'm, I'm on the same page with you.

Um, what, what do you mean exactly by the path-based formulation?

So I think it's, um, when we're now, just, just, just figure 4.3 from the textbook is kind of a good touch point.

Um, on the top is the discrete time explicit states and observations that time points are considered.

Time point 1, 2, 3, 4.

In a continuous time.

Future time points are only predicted through a Taylor series generalized coordinates expansion, which is something we talked about in, in 52.

Um, or continue from there.

Just wanted to bring in those two textbook figures and how they can be combined in, in a hybrid architecture that kind of presages what we're talking about today.

Yeah.

Okay.

It's interesting that you bring up generalized coordinates because, um, yeah, when I, when I was mentioning, uh,

myopic active inference or like not using path, um, I wasn't specifically mentioning, referring to generalized coordinates.

Kind of like the more simpler version without the generalized coordinates, because you're right, you're absolutely right that when you have generalized coordinates, um, you, you do get trajectories again.

The, the difference or, or one of the differences is in discrete states, um, you, you may have a limited computational budget that kind of limits you in several ways,

but at least theoretically you can plan infinitely or arbitrarily far ahead in the future.

Uh, even though your beliefs would get very imprecise, um, in continuous states, when you have generalized coordinates, you have this notion of, um, encoding implicit trajectories in the future.

Through these Taylor expansions, but the Taylor expansions are only by design, only going to be accurate on a very small or, or small, uh, time interval.

So, so already theoretically, if you have only finally many generalized coordinates, which is what you always have in practice,

you are now able to plan very fine the future.

This is, um, this is like much more sophisticated than, than any kind of active inference without the generalized coordinates

or the things I was mentioning before.

The, yeah, what, what, one other thing, um, is so, so when you do planning in the sweet states here, like comparing multiple trajectories of actions,

if I remember right, but, but correct me here if I'm, if I'm wrong, when you do planning in the continuous states for generalized coordinates,

so then, uh, at the bottom of 4.3, you're always minimizing the gradient of the free energy with your actions.

So even though you plan the, the trajectories of states in the future, if you did that, you don't plan for sequences of actions.

I, I believe that that's true or another way to say that is that the continuous time formalization,

even if you do have a high degree, uh, Taylor series expansion or a lot of generalized coordinates,

it isn't explicitly considering counterfactual scenarios like tree search in chess.

It's like seven moves down the road. If one, two, three, four, five, six, seven, then eight, a eight, b eight, c.

Whereas in a continuous time expansion, you get one polynomial approximator, but it isn't saying, well, what if time point 12 were different?

All you could say is something about the statistics of that path as interpolated, but you wouldn't be able to directly, um, make strong guarantees outside of following the gradients around that inferred path.

So maybe, maybe I've mistaken something, but I, either in one of the points that you've written there, Daniel, or something Lance said,

I remember the word reflexive for the continuous time action of inference. Is that, is that what we're saying here is that, um, yeah, there we go.

Without planning, um, at least with the continuous, um, time formulation.

Yes. It's sort of doing something with the expectation of what's going to happen in the future.

But as you say, it's not considering counterfactuals. We're not considering if I do this, then this, if I go 10 meters rather than 20 meters, this will happen, et cetera.

Yeah. Yeah. I think it would be interesting if he, if he has that question to Cal, um, if he, if he draws the same distinction, but, but I would totally agree with you.

Um, I would say, so at the beginning I was referring to continuous active inference without the generalize coordinates as being reflexive.

I would say that with generalize coordinates, it's still reflexive, but you're right.

That it's still, it's already much more sophisticated in the sense.

Yeah.

The generalize coordinates are like a way to bring the depth in.

It's like doing momentum trading on a stock over multiple timescales.

You're still making an instantaneous decision in the sense you're looking at a vector of the current moment and then looking at different trends and first, second, third, et cetera, higher moments.

However, again, there's not explicit planning.

So this sort of gets into the functional underpinnings of anticipatory behavior and whether anticipatory behavior can be loaded up into a sort of reflexive, intuitive moment or, and, or to what extent it involves explicit tree-based search of branching possibilities.

Yeah, totally.

Um, okay.

Okay.

But just to close the loop on the sparse, on the sparsity, could we say generative models are defined by their sparsity.

We're literally talking about some set of random variables, nodes, and their edges, which defines their sparsity conditions.

Any node interpolating between two other nodes is a blanket.

People often are interested in the agent environment blanket as interface.

However, taking the graphical model as a base graph itself, we can think about the B matrix here as being a blanket, for example, between these other two nodes.

So this is the sparsity conditions of figure three and the, I mean, whatever model you design, the connectivity is the sparsity condition.

Renormalization generative models have a special template or recipe for how the sparsity conditions are related within and across levels.

levels.

Okay.

Arun, want to pick one of these or which one to go to next?

Um, I actually think the next one is they're sort of roughly in an order.

Uh, so yeah, I think talking about the RG operators, because it is the really new thing in this paper, what makes a good one?

What makes a good RG operator?

So, um, yeah, I mean, I'm assuming if you ask this question, you probably know more about RG operators than I do.

Um, so it'd be interesting to know what you think about this.

Um, yeah, to be honest, like, uh, yeah, just to give like a comment, um, this is one of the things that I really need to read up on.

Um, Kyle repeatedly says there is this universal aspect to renormalizing giant models.

And I agree that there is some part of nature that seems to be scale-free in terms of like these marker blankets had different scales.

But to me, there is also a lot of nature that is not scale-free.

I mean, not everything that you look at is a fractal.

So in what sense are RGMs universal?

Uh, the super cool thing and surprising thing for me in this paper is that, uh, there's a lot of things that you can do with RGMs.

There's a lot of things that you can learn.

I'm not sure to what extent, I mean, I would really like to know, okay, well,

this is one thing that you cannot do with RGMs in terms of like, uh, I cannot memorize this.

Yeah.

That's something I'm not, I'm not sure about.

So maybe I can sort of come back on that.

Uh, so I think the use of the SVD and effectively principle components analysis in this paper is actually like a really nice introduction to renormalization groups.

If you don't know what they, if you haven't heard of them before, uh, it's very common in other aspects of more like traditional pattern matching and machine learning.

And there's loads of good tutorials, uh, just out on the web.

We've got a couple of links in the slides, uh, to explain what it does.

So I think just saying like to summarize that very quickly, PCA is, uh, you do a transformation on your data to maximize variance amongst particular dimensions.

And then you can do a dimensionality reduction by throwing away the dimensions you don't want.

So that really neatly leads into the definition of RGMs in this paper, which is the core screening.

It's the critical part is that you are throwing bits away that you think aren't important at particular levels.

And then as you go up levels, you throw away different dimensions, but they feed into the lower levels.

Uh, but I think it's, there's also a lot of talk in this paper about like the spin block transformation as well, uh, how the pixels are sort of blocked up and then you say, okay, we're going to core screen in space,

uh, and also core screening in time.

That's in a couple of the examples in the paper as well.

Uh, but I guess my sort of question is particularly on the, like, how do we know our core screening is not throwing away actually useful information?

And how does that relate to the data that we're putting into the training as well?

Uh, I think that's cause I know you mentioned before, this is all about learning from expert play, but are we also possibly throwing away some of the expertise when we core screen?

Totally.

Um, yeah, totally.

I mean, um, as far as things stand at the moment, this is more of a open question, a research question.

So some of the things that we're, we're doing is, you know, testing RGMs on, on different kinds of, uh, expert

play for different games

or for different environments, uh, in and outside of Atari and looking at how well it works.

Um, so my own take right now is that RGMs are pretty good at memorizing, uh, really a bunch of things, a bunch of expert play.

Um, the, well, two things, um, the, for, for RGMs to do well, the, the game or the environment needs to have some kind of orbits, some kind of, um, orbit structure that, um, yeah, the RGM would learn.

So that's like one thing that's pretty important.

Um, the main challenge of RGMs that, that we've kind of, um, been looking at or, or looking into is, uh, generalizability.

So one of the things that, uh, we show in the paper, which is, uh, pretty cool, I think is this, uh, Dove example.

So we, we learned this Dove video with the RGM and then we occlude, uh, the Dove and the RGM is able to infer,

oh, um, um, I'm actually, so it's going back to its training data, its reconstruction of the training data and reconstructing the Dove. Um, so that's one type of generalization that works very well.

But it kind of highlights, um, perhaps, I mean, what, what seems to be a drawback as well. So

the RGM, uh, learns representations of the training data, which means that when you expose it to non-training

data, so, such as the Dove, which is occluded, it will go back to training data and be like, okay,

well, this is the Dove that I saw in my training and so I'm going to reconstruct it. What if, um,

you train it on expert play for a game or expert something for something, and then you initialize

the RGM at a different condition. So let's say, um, so, uh, let's say you train the RGM on walking data, some robot walking. The RGM will learn the orbits of walking and it will be able to reconstruct the

motion and the actions of walking. But now if you initialize the RGM of like, I'm lying flat on the ground, it won't be able to stand up to go back on the walking orbit. It will infer that it's walking and

it will execute the actions of walking even though it's flat on the ground. So you can imagine the motion, um, the motion that you get out of it. Um, and this is a kind of a generalization aspect where RGMs are

really lacking is if you're outside of the training data, you're going to infer that you're in your

training data when in fact that you're not, how do you go back to the things that you've learned? How

do you pick up the orbit? Yeah, this, this definitely came up in the previous one, the point zero as well,

Daniel, we're talking about resiliency of models. Um, and, uh, this is, I think a really interesting

question of, yeah, how do you know if you're out of distribution? If you like, there are, there are

other sort of machine learning applications of maybe like kernel density estimation or whatever,

where you might say, okay, I've trained on this data, I get this new data. Can I compare it to

probability distribution and say, oh, wait, this doesn't look like anything else that I've seen before.

Hit. I'm now very uncertain on what to do. Is there a way we could apply those sorts of principles here?

Yeah, I suppose so. Um, so this is, um, this would be a very interesting project because,

um, yeah, I mean, this is the main limitation with RGMs right now. And, uh, yeah, there's possibly many routes forward, but, um, yeah, the moment that's the main challenge. So taking inspiration from those

things that you mentioned would be super promising. One, one way that, um, active inference models at least have a diagnostic is surprise bounded by variational free energy does give you a measure of overall how likely something is apt to be. So if you have a, um, training on driving safely, then you're minimally surprised by the safe drive. You have a measure, at least this kind of threat agnostic measure of something different is happening. However, it's almost a general information theory problem of, well, how the space of what you don't know is always going to be so much larger. There's always more that you could be surprised by like for all the combinations of valid English sentences that language models are trained on. There are more combinations of gibberish sequences. So then how, how to make something that can weave its way back into the orbital zone or into a zone of control. And that's, that's where some of the multi-scale architectures and switching architectures come into play because the, the variational free energy value itself isn't going to say what to do when the car is off road, but at least it would be a diagnostic of how, like a kernel density estimator, how similar it is to training information.

Just a, another thing to bring in as well. I think we still bring all the questions in at once, but, uh, it's all, it's all related and it's all relevant. Um, the question of how, if we are only training these things on expert play, uh, how can we create resilient agents that can recover from unseen positions to, so quote unquote, get back on the path. So I think one of the things that came to my mind when I was reading the paper was if you give it data where it starts off in a losing position and you can show it ways of going, ah, if you are losing, this is how you get to winning. Then that would become a new potential attractor set that, okay, if I am losing, I can then move to another sets where I'm winning. So you explicitly give it that journey as data that it sees. But then I think we come on to one of my other questions, which is how do we then tell an agent that has learned the structure from the data itself, which attractor sets are good and which are bad. How do we inject value into these to say, okay, we've given you this data where let's say the self-driving car is a really nice example. Driving safely is great. If you are in an accident and you drive off road, this is how you get back onto the path. But if it's learned the structure of those actions from the data itself, how do we then inject the value that actually driving safely is good and being operated is bad?

Yeah. I mean, yeah, really interesting questions. So I have a few thoughts that I think echo what you said and I don't have specific solutions, but just echo what you said. So for example, in the case of walking, we might ask, okay, well, if we have the training data for walking agent and somehow the agent is like down, either sitting or lying down, we should teach it how to stand up. And then once it's recovered, it's starting position, we can go back and use the RGM policy. So that's one solution.

Yeah, going back to the starting point of the orbit that works. But there are challenges. So for example, and this is maybe a slightly different challenge, but also coming back to self-driving cars.

A self-driving car is a very, very high dimensional system. Lots of moving parts, actions, things that are going on. So if you give the RGM a lot of driving data of driving safely, I'm assuming the RGM would learn comfortably. But now if you expose it to a new situation it hasn't seen before in the training set, and I would assume this would just happen very easily, right? Because we're

such in a very, very high dimensional space. And when you think about it, it's kind of a personal dimensionality problem. Because you're feeding trajectories to the RGM. So trajectories or Orbeez, these are one dimensional objects. And you're in a state space that can be very high dimensional, for example, in a self-driving car. And what you want is to get full coverage of like the underlying manifold of state driving. And what you also want is if you're in an unsafe driving situation, you want the trajectory that gets you back to the manifold of self-driving. So you have these two problems. And, and yeah, so one problem coming back to the curse of dimensionality, is that if you have this extremely high dimensional manifold of state driving, you want to cover it with one dimensional trajectories. The higher the dimension, the harder it's going to be because you're in a trying to cover a large n dimensional thing with one dimensional objects. So computationally, it's going to be a problem. And yeah, so there's this generalization that doesn't work like within, as you said, like within data distribution generalization, that's a problem. Because if you give it a lot of data of the driving safely situation, we would want it to kind of extrapolate over all safe scenarios of driving. And we don't, we don't really see that at the moment, or at least maybe it happens, but it could be a challenge in some situations. And the other one, which is maybe the more fundamental problem is you have this huge region of driving not safely. How do you go back to driving safely situation? Also coming back to the example of walking, how does that happen? And and that's also very hard, right? Because I think you really want to, you know, have a trajectory back to the safe situation from every possible starting location. But as the state space becomes very large, that also becomes very difficult.

Yeah. When thinking about all the angles, your joints could be in all the muscle energies, there might be a very small zone.

So how, how to really think about adaptive active inference agents at the agentic level, and then also more at the systems design level, how could components be robust and adaptive and simplified components doing better on a subtask, and then design a composed system where the sort of low energy efficacious operation of the smaller units is designed in a way where they are not dealing with that massive overall computational challenge. They're dealing with local computational challenges. However, that moves the design challenge to, or moves the the functional challenge to design time rather than inference time. But most morphologies don't walk. But if you have one that walks through the revolution or whatnot, then it may be a relatively simplified inference problem at runtime. However, the challenge is basically figuring out which of those sparsities and message passing schemes would result in tractable node level inference. Otherwise, it would be hard to do on a relevant hardware.

So that's the way that we're going to do that.

And adding to that, if I may, so going back to the walking example that you mentioned, and also the generalizing within data distribution. So there is evidence, and this is also something that we've kind of encountered empirically that walking is a chaotic orbit, which means that if you...

So let's say I'm here, and I'm going to take some actions, and I walk. Now, let's say I start from a

slightly different, infinitesimally different initial condition, and I take the exact same actions, I'm going to fall at some point. And I just take the chaoticity, right? So if you take the same actions, but you start from a different initial condition, you're going to diverge very far away at some point. And so we have, I mean, I think we have this problem in RGN that if we teach the agent to go back to the standing position, and then take the RGN policy, if the agent has a slightly different initial condition than what the RGN was trained on, the RGN will infer that it is in the true starting position, even though it's slightly different. And then we'll take the actions of the RGN, and it will fall at some point. So you have this idea that just memorizing just doesn't work in that situation. And yeah, that's also an issue.

But maybe just a small question there. Is the issue there with the coarse graining part, in that a slight change in initial conditions is being thrown away by the PCA and throwing away dimensions early on? And then when we go back and do the inference, we infer that we are sort of in quote unquote, the same place, because we're thrown away, like something that we've is actually signal has gone to the noise part of this.

Yeah, I imagine that's one way to look at it. The way I've been looking at it, but I haven't been thinking about it in those terms, is just if your training data is one particular, you know, orbit trajectory, and you start slightly different, because you're inferring where you are based on the training data based on the structure that you've learned, you're going to infer that you're on your right trajectory, even though you're not, and you're going to reproduce that, but you're going to land arbitrarily far. But yeah, I assume it's the same phenomenon.

This kind of gets to the whole structure determination question. Chaotic systems are sensitive to small changes near critical points, but then there might be another zone of the system that isn't so susceptible to critical points. So like, let's look at the chaotic attractor example from the paper in figure 13.

If we were to make a small change while we were out on one of the arms of the attractor, it might not change things that much. But then there might be some other points where it does lead to taking a bifurcation. So if you can break up systems so that the coarse graining captures those bifurcations into two discrete categories, then it discretizes a continuous chaotic system well. But then if you are applying a grid that's in commensurate with what's actually happening, then any grid you draw is going to include trajectories that have radically different outcomes. So the overall predictive efficacy or the variance retention of the SVD is going to be low.

Like if there's actually two zones in a city and they're one mile squared and they have different properties and you have a one mile grid and you actually capture it, you carve nature of the joints or articulate it at the joints. But if every block has mixed zoning and all you have is a one mile grid, you could do the best placement of your one mile grid, and you could do the best placement of your own grid. But it still is always going to capture something that's kind of mixed. And this kind of related to a few of our thoughts and questions like, how do we determine what window to use just from a generic perspective? Like there was 10 levels

in the discretization of a grayscale image, or there was a 4x4 or the MNIST images were pre-processed to 32x32 pixels

and then they were blocked in 4x4 grids. So how much

sweeping or how do you do inference on the number of discretizations and the window size and all these other features? Like what kinds of trade-offs computationally do you look for? Or how many levels of meta modeling do you feel like are relevant to engage in when you could sweep across discretization of N and windows of M ? And then you could have, well, window size of 6 with a discretization of 3 gives this performance. But something, so how do you even navigate through that combinatorics?

Yeah. Yeah, really good question. So yeah, at the moment it's very much, I mean, because this is very much a new thing, for the moment these are hyperparameters and we're very much in the exploratory phase of,

okay, well, this works and this doesn't work.

I mean, I'm assuming for, I'm thinking for the paper there wasn't much hyperparameter tuning, like it was all, you know, powers of 2 that were chosen and a small number of layers.

But when you think about it, I mean, you could do renormalization by powers of 3, powers of 4,

4. You could choose to have smaller or bigger patches, increase or decrease the number of patches. So, there's many things you can be looking at. And I would say that's also a problem that you would get if you were to do structure learning with a generative model that are not specifically renormalizing terms of models. You also have that problem of, okay, how many patches, how many layers and so forth.

So this is not a problem that's like specific to this approach. What this approach has,

which makes the problem a lot easier is that there is a lot of interplay between the number of layers and the number of patches, right? So if you choose, I'm going to renormalize by 2. So it means that two states at one level become one state at the next level and two states at that level become one state at the next level. This gives you a relationship between the number of states and the number of layers that you have and also implicitly the number of patches that you have. So you have this like big constraint that comes in renormalizing terms models that makes the search a lot simpler than otherwise.

But I think ultimately what you would probably want is to infer, and I think we hinted at that, infer like the number of patches and so forth. Yeah, we're not doing that. It would be helpful to do that.

I think another possible thing we could try is maybe start with a budget of this many layers, patches, like basically combinatorics and set. Okay, you have this much computational power for understanding this data. Figure out the best way of allocating that, because at some levels, you might have more variability than others. And maybe you want to allocate your computational budget at the higher abstract levels than you do more so than you want to do at the pixel level. And perhaps we could learn that from the data itself, given a constraint of this many parameters. And I think you could probably code that up multiple ways. But one way you could have priors on those hyperparameters. And then I think as Daniel suggested in the point zero, do a free energy calculation on that, compared to that your priors on those hyperparameters and just minimize free energy that way. But you could also probably do a hacky approach as well, and just go, well, what gives me the best performance, call it that.

Daniel Ponga, Ph.D.: Yeah, and just to kind of echo that and complement that. So, as you both mentioned, the issue is, well, there's so many hyperparameters. I mean, there's so many quote unquote, right? Because if we're using GENT models that we're not, we don't imagine there would be even more of them.

Daniel Ponga, Ph.D.: But even here, we have a lot of hyperparameters. And so that's a separate thing that we're also working on, which is, can we infer the structure of the model? Instead of just saying, oh, well, we have this data, this is the model that we get. I think we need to be recognizing the fact that with limited data, you're always going to be unsure about the resulting model.

Daniel Ponga, Ph.D.: And so we need to have probability distributions over models.

Daniel Ponga, Ph.D.: And so one instance of that is what you mentioned, like we have probability distribution prior over the number of patches, and we optimize that with respect to free energy.

Daniel Ponga, Ph.D.: But we could also have probability distributions over three layers or four layers, and so forth. And I think this is where we, this is kind of

Daniel Ponga, Ph.D.: I mean, yeah, beyond this paper, but where we need to be going is having uncertainty about the model. And so then if I have uncertainty about the model, I can take actions upon the world that maximize the expected information gain about the model.

Daniel Ponga, Ph.D.: So I'm going to get data that's maximally informative to refine my beliefs, ultimately converging about the best model of the world, or maybe a set of candidate bests, if there is a fundamental instruction where you cannot find which one fits best.

Daniel Ponga, Ph.D.: But the point is,

Daniel Ponga, Ph.D.: In this approach to structural learning, and I think in every other approach that we have in active inference so far, we're not recognizing the inherent uncertainty about the model.

Daniel Ponga, Ph.D.: And so what we're doing in the

Daniel Ponga, Ph.D.: yeah, in all these structural learning approaches is

Daniel Ponga, Ph.D.: is a kind of, well, imagine if you have a prior of the models, and then if you have data, you have implicitly a posterior of the models.

Daniel Ponga, Ph.D.: Now, this is what I think we need to do, and what we're currently doing implicitly by taking the model with the lowest free energy when we actually do that,

Daniel Ponga, Ph.D.: is taking a flat prior of the models, and then taking the most likely model in the posterior. So we're doing this maximum a posterior estimation when we're optimizing model free energy. And yeah, ultimately, we want to have a distribution of a model, and also recognize that a priori models are more likely than others. Daniel Ponga, Ph.D.: That's funny about having a flat prior over models. It's almost like return of the frequentism. All models within this class are equally likely, so that's the special case of Bayesianism where it sort of anachronistically is frequentist again, because then you're selecting the maximum likelihood model structure or weighting model the model. So that's the total reliance in a portfolio on its maximum likelihood estimation.

But then rather than using an ML value, the model itself is actually engaging in a Bayesian inference process. How about representations of uncertainty within the model? Like, just taking a given architecture as fixed. I'm curious about how are uncertainties represented? There was some discussion

of lossless and lossy compression. And do we have any way to have interpretability of the uncertainties across layers? Like, do you find that you can trace those state estimations and uncertainty estimates and they ring true? Or once we're past the pixel level, is through the SVD and all of this, are we already in spaces that are not human semantically interpretable?

Daniel Ponga, Ph.D.: Good question. So yeah, coming back to one of the points that Arun made, because we do this SVD and we throw away some less relevant components, we are throwing away information.

And so we're always in this lossy compression kind of setting. Now, what we've done in this paper and also in other experiments is to just try to reconstruct expert play or any other kind of video or agent from the RGM. And usually it works pretty well. So we, as you can see with the DAOV or with many other examples,

the RGM does a good job at compressing and reconstructing. But we haven't actually looked into mechanistic interpretability of like this state in the RGM actually encodes this and this state in the RGM actually encodes that. So that would be interesting to look into.

Daniel Ponga, Ph.D.: I think in the MNIST example, there is a little section where it said like, okay, at the very top level, we say there's basically a value per class of digit, which feels very much alike. Okay. In the intermediate stuff, it's very uninterpretable, but at the end, we're going to have, okay, we're going to say each of these values at the top level can have corresponds to a digit, because we're designing a classifier. I think it's a really interesting question that's not just specific to this paper or to Active Inference. I think it's a self-supervised learning in general, which is how do you interpret that if you're learning the structure of data from the data without anything labeled? How do you get that interpretability? I don't think there's an easy answer to that.

Daniel Ponga, Ph.D.: Yeah.

So here at level four, okay, so this also notes, the matrices in this figure are not simple likelihood mappings. They're concatenated likelihood mappings from all hidden states at one level to all hidden states and paths at the subordinate level where the sum to one constraint is applied to states each child could be in. So we're taking some higher level path trajectory and pulling lower level observations from a probability distribution sum to one at that lower level for the child. So here at the top level, there's 10 states. So it's almost like if you came across an alien handwritten text, how would you know how many letters there were? Or if there were several writers with different styles, how would you say how many discrete symbols?

Daniel Ponga, Ph.D.: Because that's kind of like this unstructured or semi-structured learning question. You have cursive handwriting from a language where you don't know how many discrete symbols there are.

Daniel Ponga, Ph.D.: So you know there's a discrete compositionality at the level of the word, let's just say. However, you're getting the raw data and then the pixels to planning in the linguistic case, or the phonemes to planning,

Daniel Ponga, Ph.D.: is like how do you go from a continuous visual or sound data to inference and planning over language production.

Daniel Ponga, Ph.D.: But then you don't know how many discrete elements to have

per se. But if you can have a, if you can be making a domain specific model, that may not be a relevant question. You may just be able to say, we know we're going to get to 10, find me the best 10.

But it's interesting that the compositionality of the RGM could extend to higher orders of structure

Daniel Ponga, Ph.D.: with the trade-off that there's just more and more as you continue to expand how many states there can be.

Daniel Ponga, Ph.D.: It doesn't escape the curse of dimensionality.

Daniel Ponga, Ph.D.: It's like swimming in the curse of dimensionality or casting the spell of the curse of dimensionality,

but at least in a way where the model architecture is going to be able to be implemented using

Daniel Ponga, Ph.D.: The inference methods that are the same across different scales.

Daniel Ponga, Ph.D.: Quick question on that. Do we think that the PCA part of this could give you a clue as to how many values a particular state could have in a layer?

Daniel Ponga, Ph.D.: I think Daniel, you had this idea in the point zero of saying, well, maybe we just have as many values a state could occupy that explains 80% of the variance at that level.

Daniel Ponga, Ph.D.: So then you could say, then, I mean, we're sort of moving the complexity from this part into construction of the training data, which we should definitely talk about at some point on this call. But you could then say, yeah, exactly. That's the graph that I wanted. If we're going to just a priori say, I only need to explain 80% of the variance on this stuff, chuck some training data at it. And then we say, ah, this training data had like 10 digits. And then you basically end up picking 10 values because that explains enough variance in the training data to do a good enough job.

Daniel Ponga, Ph.D.: Yeah, for sure. And I would say that's also basically what happens when we're optimizing model free energy, right? Because we're trading off accuracy and complexity. And what you're mentioning, maybe like a more deliberate way of saying, well, we have this target. What is the minimally complex model to achieve this accuracy target, for example? And yeah, yeah, that would be super cool to do. I think specifically for more like engineering and industry applications, that would be something that people would be very interested in because,

for example, regulators, they don't talk in terms of, oh, your free energy should have lower than this value. And then you can use your model for self-driving cars, right? You're approved.

It's more like, oh, when some kind of accuracy or some kind of explanation of some data about a third threshold, then yeah, it would be very worthwhile for this kind of application.

Daniel Ponga, Ph.D.: Yeah. How to convert from a sort of model intrinsic currency, the way that different structural models are compared with each other, or the way that alternative parameterizations are compared within the same active inference model through comparison of their free energy values, how to convert that into a value or a currency that matters, like an actual risk profile or a cost or a time estimate. That's the critical question. How does the reliability of the inference translate?

Daniel Ponga, Ph.D.: What would be an interesting way to go, Lance? And then for those in the live chat, please feel free to write some specific questions that I'll ask.

Lance Stavrosen, Ph.D.: Sorry, specific interesting way to go for a way.

Daniel Ponga, Ph.D.: Or just what do you feel like we should continue with or...

Lance Stavrosen, Ph.D.: Oh, yeah. Okay.

Daniel Ponga, Ph.D.: How, also, how do,

how do others pick up and continue with this work beyond studying the background 11 and a half Ph.D.s needed to continue with this, knowing that better and more advanced open source tools are under active development here and elsewhere. But I mean, what's the way to engage with this material, with the current state or what are your next plans for how it continues from this preprint form?

Lance Stavrosen, Ph.D.: Yeah. So good question. I think there's already a lot of ideas that have come up in this poll. I think my interesting idea is what Arun mentioned, like using tools from machine learning to improve generalizability in RDMs. More broadly, my personal take home message, because we've been talking a lot about, well, first of all, how our RGMs work, but also what their limitations are. So this is my understanding is that this is a very new kind of model. And there are a lot of limitations. But there is the, like the recall thing about this paper, and this is why I'm personally excited about RGMs, is this universality claim. The fact that RGMs are super expressive, you have this like one architecture that can learn many, many things that are pixel based,

as we've seen in the paper. And so this is what we've seen empirically here and in other places.

And there's the more theoretical reason for why that's the case. And I would say that's more of a physics question of like, understanding renormalizing groups, and why the world may be or may not be renormalizable to

some extent. So I think it would be super cool from a theoretical perspective to have a better understanding of that from an applied perspective. So we've seen all these limitations. And it was already clear on this call that for each of these limitations, we don't know exactly whether it's a fundamental limitation, whether the limitation lies from the current implementation, or maybe from the current instance of RGMs, that we can modify them in some clever way to actually overcome the limitation. So there's many different places where improvements could come. Third, the actual structure of the thing, or maybe even the theory, maybe we need more than renormalizing different models, maybe renormalization is not all you need, you need something else. So at the moment, I think we should be, you know, looking at where each of those limitations are, and you know how to go forward from there. The other thing is classical active inference, like the discrete planning, is does actual planning, you know, it plans multiple steps ahead. And it has this counterfactual aspect. In RGMs, you don't have that. You just learn the transitions from state to state at each level, and then you infer them from the data. So you're just like, you're like in this self fulfilling prophecy mode, where you've seen your training data, and you're reproducing that from the RGM. And it would be cool to know if you can combine the strengths of classical active inference and the RGMs. To have some planning at one or multiple levels, some actual planning with information gain, to actually help this generalization aspect. By improving the model with some, you know, new relevant data. So there's many, many, you know, directions this work could be taken in. Personally, the direction I'm most excited in is this idea of moral uncertainty. Because in, so, I mean, a big

question in machine learning, and also developmental cognitive science is this idea of structural learning, how do you learn the structure of your model of the world? Now, there's a lot of papers, evidence, and I think there's also a lot of debate. But one interesting paradigm is this idea of model inference. And the way we've been thinking about structural learning and active inference so far is about optimizing model free energy. So we take the model that that has the lowest free energy. When we do that, we are frequent, we are Bayesian in the states and parameters, but we are frequentist in the model. And that goes exactly to what we're saying before, this idea that you have this flat prior, and you're doing much more posterior estimation. And so we know that being frequentist is fine, but you can, this means that you can overfit. It means, yeah, basically you're selecting the model structure that overfits, that best fits the data and which may overfit. You're not recognizing any kind of uncertainty. And there's actually a lot of theorems that show that with limited data, but even unlimited data, there's many instances where multiple models could explain the data equally well, which means that they would have the same free energy. Now, which one do you pick? And two different models could yield two different decisions. So if you're in a real world scenario, one model may be, you know, the true model or what we want the agent to have, the other one would be wrong and could have some, you know, unsafe decisions being made down the line. So what we want to be moving to is this idea of full model inference, not just map estimation, but actually having a full posterior of our models. And this would be, for example, a particle approximate posterior where you're entertaining a mixture of models. And so basically means that we would be doing basic model averaging for all of your predictions. And yes, we want to be moving towards that, in my opinion. And what this affords, which I think is super important, is so when you have this uncertainty about models, you have this idea of taking actions that maximize the expected information gain about your model. And so now you can start to learn your model in a self-supervised way. And hopefully in a way that also generalizes, because you're making this epistemic search that informs your model, which we're currently not doing. At the moment, we're seeking data that's informative for the states and parameters, and we're overfitting the model and we're going to be moving beyond that.

Awesome. Yeah, very interesting. It's like equivalent to only doing policy selection on pragmatic value is just utility seeking. So then if the utility of the modeler is model fit, they have entered into a pragmatic only analytic mode that in single-minded pursuit of what appears to be a totally valid, defensible, analytic goal of accuracy or of data poster posteriori fit, ironically produces local overfitting. So it's like exactly why we use expected free energy for policy inference, because it does have the epistemic value component. And then now with this level among portfolios of models, it's like, we need to remember why we put the epistemic value into the architecture at the base layer, because it isn't enough to just have the epistemics at the base layer and then forget about it at the modeler's level. There's actually a role for the epistemic value and also for the lifelong learning. Whereas in the examples here, there was a training phase and then a test phase, as opposed to like the rules of the game, like sleep-wake type train and test and updating with online inference, like if the rules of the game changed or something like that.

Yeah, totally. Yes, you want to be able to handle environments. Let's say, coming back to games, because I guess we love games, but it doesn't have to be games. Let's say you're in an environment, and you learn how to play the environment, you learn the genital model. And then you get to the end of the level, you get to a new level, and the environment changes somehow, the number of categories changes and so forth, you want to be able to handle that in a graceful way. And you want to be able to handle the inherent uncertainty about the world. So there's a lot of work on AI-driven scientific discovery. And you can argue that, I mean, science is about understanding the mechanisms that generate the data that we have. And in particular, the mechanisms of cause and consequence, right? What causes what among our latent states. So this is like, a totally, I mean, like an example that all of us can relate of the fact that we have uncertainty about causal structure. And we may also have uncertainty about the number of representations used to explain a particular dataset. So let's say I'm a young child, or I'm a zoologist, and I'm in the forest, and I see an animal I've never seen before. Is that animal part of an existing species that I know? Or is it part of a totally new species? And there's two hypotheses. At this moment, I don't have enough data to decide between either of them. I'm going to conduct as a scientist experiments. Well, just going back a little bit, it means that I have a posterior with two different models that are equally likely or somewhat likely. And then I'm going to conduct an experiment to decide whether this animal is part of an existing or new species. Which means that to do that, I need to have some notion of what would happen if I did that, like information gain about the model that I'm using about the world. This is like a super simple example of like, we fundamentally need that in a, for one in a biomimetic agenda of like creating agents that, you know, resemble us and modeling and so forth. Also from a statistical perspective of like not overfitting. But it's like pretty hard to do in practice. How about coming back to one of Arun's questions. In what ways is time a special dimension? We talk about planning through time and that's intuitive. From a folk psychological perspective, how does the model handle time similarly or differently about RGMs or just in general? I mean, what, what, what is time? How is it being treated similarly or differently to other dimensions? Um, yeah, good question. I think it's a very fundamental and also physical question. Um, um, but just in terms, I mean, very, very simply in terms of the model, time is the only direction that we can, uh, control. Is the only, I mean, we can't control the flow of time, but we can control the evolution through state space as time unfolds. And those are actions. Now, uh, one interesting thing about the paper is that we not only renormalize in space, in state space, but we renormalize in time. And so we have this, uh, in the current architecture, this one step ahead actions at each level, uh, time horizon of one, it could also be two or four or whichever you want, as long as this is the same as at each level. Um, but yeah, the point is we renormalize both in space and time. And, uh, I would be, I think this would be a good question for Carl of like, why is it important to renormalize both in space and time? I don't really have a sense for, uh, why that is, but he probably has a very compelling reason for, for what that is.

So I'll give a little bit of detail as to why I asked the question. So there's a very technical reason. And then there's like a wider reason. The very technical reason is to do with the overlapping

part that we talked about before. And I noticed in the paper that doesn't seem to be, there doesn't seem to be overlapping time, uh, uh, sort of factors you go from one state above to generates the initial state at the level below, and then a transition. And then you do the next pair, but the pair of observations at the level below don't overlap. Whereas in the MNIST example, you had, uh, there were radial basis functions that were like four pixels wide or whatever. Um, but they overlapped, which I thought was really interesting. So that was the very technical reason why I asked, oh, is time separate here when the pixels don't overlap as you're doing the renormalization.

But I think the wider question of why it's interesting is that I don't really understand how you would plan with respect to space. But as you say, like with time, it's, it's the rollout. That's the critical dimension of you go, if I do this, then in the future, something happens.

But I, uh, my, my brain doesn't compute what planning would look like in space.

Uh, but yeah, we will definitely ask Carlin the dot to, and probably be just as confused at the end, but that kind of makes me think of a wave propagating out like in a game of life situation, a wave is propagating out at one, one click per time step. So it's moving. It's a wave front in space time from the surfer on the wave. They're not moving, but then re-normalized over space. It's like, what will happen to that time step at the next time step? Well, if it's, if the wave's going to hit it or it's going to leave, but from the waves perspective, re-normalizing through space time, it's stationary. It's a stationary traveling wave

that still is treating time in a little bit of a different way than space.

But for all systems that we're studying in time, which basically are the natural systems, we can't really get away from it. But maybe there's like a hyper time crystal view on some of these systems that spatializes time.

So, yeah, in some of our last pieces, again, if anyone has questions, feel free to write it. Okay.

Rex asks, how has the progression been with the Atari challenge? Encouraging results?

Yeah, I mean, really good question. I'm not sure how much I'm able to share, just because of confidentiality issues. But I would say, overall, it's quite challenging. Definitely very interesting results. Many different approaches that are being tried, because of course, you could, I mean, you have this RDM approach, but there's many other types of generative models that, you know, you could be exploring for solving this type of task. The really interesting question is, what kind of structure or structure is sufficiently universal to be able to express any kind of Atari game? And the day we solve that, we'll actually have learned a great deal. At the moment, we have some leads, we have some promising results, but we are still far away from being able to solve all the games with the same structure. So I think there's a very interesting progress in terms of scaling up. But yeah, so still far. And as always, with these research questions, it is always hard to know, you know, how long we'll need to actually solve the challenge.

Yeah, maybe just think of the game, how children pick up games, and Calvin Ball making up games, and what cognitive capacities support the idea. Like, maybe there's a game game, hangar, or high level game virtualizer, that then can be like a sandbox for creating games with

arbitrary structure. You know, you're going to tell me a number that ends in a seven, and then I'm going to do this dance. It's like, that's our game. But because it's a rule set within a space, within a container that's understood. And so, although generalized coordinates, RGM, and so on, in theory, can go for infinite number of dimensions. For any finite system, it may only be several levels of generalization, or path extraction, or just in the example of time. Even with two time steps per level, at some point, with time doubling, you're talking about a transition that's longer than a lifetime. So, it becomes a kind of an extraneous inference. And similarly, making it finer scale, you're going to quickly get to the state transitions of walking at a picosecond, which again don't really matter. But if there were either automated ways, or interpretable human in the loop type ways, to say, okay, the SVD is looking pretty, pretty, pretty, pretty uninformative at this small scale, and at this large scale. Here's the bulk of the meso variants that we're working with. Here's the information content across these coarse grainings. Here's sweeping across different windows and grainings. This looks like a good, a good graining approach. Here's where the meat is to chop up. And that relates to the focusing, the computational resources on where the information content is. Focus the resources on where there's information to glean, and then truncate the model's dimensionality beyond the point of diminishing returns. Like, it almost, it's, it's, it goes without saying, slash is hard to say sometimes, but that's an extremely different paradigm for fitting small to very large models than coming with a fixed number of parameters of a large model and a fixed data set as much as you can, and then colliding the two until you can optimally reproduce the training data. While there is this element of parameterizing it on training data, in principle, you can just specify generative models and generate synthetic data. We do this all the time for different models. It doesn't, you don't have to train them on anything. You can just make a thermometer and a synthetic room and just let it play. So it's just interesting how with these new Lego toolkits and new composabilities that can deal with data and play in that sort of empirical test train mode, that they also lend themselves to a lot more dynamic reconfiguring that I think is the main piece we've been wrestling with, which is it gives us more affordances in building and modulating these models. And so, I think that's a lot of the models. But every degree of freedom in the modeler's toolkit is double-edged because it expands the combinatorics of what is possible, possibly diminishing the proportion of good solutions. But maybe those good solutions get better and better, though they're rarer and rarer. So, I just want to come in on a quick point, which I think you've talked about at the very high scale, nothing really happens across, you know, beyond a lifetime at a very small scale. You don't want to consider walking on picoseconds. From a practical point of view, I guess that's going to be very limited by your training data. So, for example, on the spatial point of view, what does a pixel mean in your training data? And presumably you have to normalize across all your different samples to be

a pixel means the same distance in an image, regardless, potentially. Or do you have to do some sort of pre-processing to mean that you're only ever taking pictures from the same camera at the same distance for that to mean the same thing across the different bits of your data? Or if they are different, does your model then have to learn that from a structural point of view? And then going on the time one, I think that's also quite interesting, maybe it's another reason why it's different, if you're introducing videos of different lengths. So if you say, okay, here's some data where in this particular video, it's 100 frames and this one is 200 frames, there's different levels of re-norm, you're limited in how far up you can go if you're just going to continually pair and pair and pair, if you've only got 100 frames compared to 200 frames. So there could be very practical considerations here for the number of levels you can have and what timescale you can operate on. But then also, I think it does beg the question, at the very top level, if you've only got, I think, for, let's say for the music example in this paper, you've, if you've got a length of music, and you, it breaks it down, like it was jazz music, and you break it down to like, notes and bars and potentially could go up to a movement, but you can only go up to the maximum level, like length of your, like a piece of music that is, that you're, that you're exposing in the training data, you can't go a level above that. And that sort of like, highest level thing could only really have like one value, so you might as well get rid of it. So there's some, there might be some redundancy at the very top there that you just sort of can always implicitly throw away.

Yeah, I think so. I mean, when, yeah, and when we have the current architectures, I mean, we always show the, the top level as having, for example, one state or, or two states with a transition to another two states. But then implicitly, you have this, even higher level that has one state, and that static, but yeah, somehow he, I mean, it doesn't do anything. So yeah, there's many things, I guess, many questions and many things that can be explored. The other thing, another question that I have is, so intuitively, let's say we're thinking, okay, I need to go to work, I need to take a certain amount of actions to go to work. Intuitively, when we plan in that setting, there's several actions that happen in the same, at the same level, or arguably, maybe not everybody's like this, but it's like, I need to go down, I need to, you know, take my car, and then I need to go up the elevator, at the workplace, and so forth. It seems that this plan maybe happens at the same level of abstraction, or like, there's many cases where we plan on the same level of abstraction. And the current RGM architecture, we just have one step ahead, or like a fixed amount of steps ahead, that are reproduced at each level. So, yeah, I mean, it, you can definitely reproduce the example that I talked about by saying, okay, well, we're going to plan, you know, 10 steps ahead at each level, and then it would fit with more complex plans. But, you know, how can we allow some flexibility there? Or is there some flexibility that we need for the time horizon at each level? Should it be different from one level to another, and so forth? Yeah, well, it's like, the controls in the studio are being explored and fleshed out, because you could sweep across different planning horizons, or you could plan for the same amount across horizons, like, just like, and it's brought up with the when it's

connected to the convolutional neural network. It talks about how the special case where all of the where the likelihood mappings are conserved over groups, one has the discrete homologue of convolutional neural network. So if you're just trying to make the best single filter to pass over an image, you have a CNN. Whereas if you open it up to different locations of the image having different filters, that's the blessing and the curse. Now you have that many times larger of a state space, but for problems that have that statistical structure, that would be the way to go. So that's what motivates this meta modeling question of how would we recognize whether we're in a situation where single filter is better than n number of filters. Like, these are where the modeling questions quickly rise in a way, because where our inference hits the metal, we know how that's going to happen. In terms of Bayesian graphical model and variational free energy, it's just how to know what level of abstraction to enter into the problem and really what's the purpose of the model. Because it's really easy to set up a situation where the, for example, policy inference is a slam dunk. And it's easy to set up situations where the policy inference is hopeless by design. So what is that sort of Goldilocks zone where if you could curate only the expert gameplay examples and do resemblance modeling, but if you already knew who is expert, then you already have those traces. So it's like just, it's interesting how with a strong floor and grounded in a really low level way to balance accuracy and complexity, we really quickly hit these data science questions about what the analysis is for and what structure the model should be taking on. Okay, in our last minutes. Okay, another question. Are any other benchmarks being looked at? Or just what kinds of benchmarks do you think for the open source community would be interesting for us to pursue?

With active inference searching and with RGNs?

With RGNs? So yeah, I mean, there's several things, right? So there could be imitation learning benchmarks. I think this would be very interesting to go into because arguably this isn't, I mean, the current state of RGNs, it's an imitation learning tool.

It's not really a planning tool. It's also a compression tool.

In terms of, so there's a tire. But arguably, if we really want to solve a tire, we want to put a planner in the RGNs.

So we just, we don't want to be able to learn necessarily from expert data, we want to learn in a self supervised way.

And so the very things that we're exploring there, extensions of RGNs in various ways.

And then, and then, you know, I mean, there's, there's so many benchmarks out there.

One thing that's interesting for going to Atari is mini Atari.

And then there's all the robotic benchmarks that I think are quite interesting.

I mean, you probably know many, many other interesting examples that, yeah, I think these are the most interesting ones, at least from my perspective.

It also depends on the, on the feature that you're developing, right? So maybe RGNs are particularly good for imitation learning.

But if you're in a multi-agent active inference, then you probably want to look at multi-agent games or things that require cooperation to, to solve certain types of problems.

Yeah, I would say it's very much open-ended and depending on, on the strength of the particular approach.

Okay. What are your next, uh, moves? I mean, how will your year proceed and what are your, what are you excited for?

So I'm super excited about, um, this model uncertainty thing.

Um, this idea of, uh, how, how do we do based on principle models in a way that's fast, fast enough so that we can plan.

Uh, to take actions that improve our model over time.

And, um, this is also interesting, I guess, from, um, bridging multiple fields together, because when we look at, um, for example, the work in computational cognitive science,

um, by, by folks like, Josh Tenbaum, Sam Gershman, they, they think very much in terms of inferring the model of the world, having a distribution of a model of the world.

It's not something that we're doing again at the moment.

Um, and I think this is a limitation.

Um, when you look at other folks like, uh, Joshua Benjo for interesting AI safety and also AI driven scientific discovery, they also have this uncertainty about models.

Um, and there's Bayesian inference, which is, uh, very important and interesting.

So then when you accept that this is, uh, the way that things need to be done, then there's the question of how do you infer, how do you do the Bayesian inference in practice?

And there's, uh, and there's a wide set of tools of, um, for doing approximate inference of a model.

It's still relatively young field.

So there's lots of improvements to be done there.

Lots of, uh, hardcore statistics and machine learning, very deep questions.

So this is like one direction that I think is interesting to pursue.

And, and then once you have that working well, and by working well, I mean, you want a method.

That's able to infer models very fast.

It doesn't need to be super accurate.

Uh, when you think about humans, uh, we typically only have, uh, you know, a few hypotheses about the core structure of something or something in the world.

So we don't necessarily need to have a super complex approximate posterior of a model.

Also, that would be very computationally complex.

So we want to look for, uh, things are fast and that operate online.

And so once you have these kinds of methods for inference about the model, then putting that into an agent and having the agent learn to know,

um, you know, in a, in a self supervised way, uh, in, in its environment.

And, and the other challenge for that is, uh, because you're doing model inference, there's no, it's not strictly speaking, it's not model growing or pruning anymore.

You're just doing model inference.

So your space of model is fixed from the outset.

Um, and so therefore, if you want developmental agents or lifelong learning agents, you want a space of models that can pretty much explain anything the agent would ever encounter. Or you want a space of model that is able to express anything that the agent would need to survive. Um, so you don't necessarily want something that's doing complete, but you want something that's, you know, a class of model that's pretty expressive.

But it should also be sufficiently coarse that you are able to do efficient inference on it.

So you have this trade off, which is very difficult to navigate of, um, trying to find a class of model that is universal in some sense,

but also sufficiently coarse that you can, you know, do things, uh, manageably.

And I think this is where, um, RGMs can potentially be very fruitful, um, because they are very expressive.

And they're also, I mean, they have very powerful inductive biases, this like renormalization aspect that makes your model space a lot smaller than it would be with other methods.

So this, uh, yeah, this is a one way forward.

I'm like particularly excited about.

Cool.

Yeah.

Expert learning is kind of the inductive side and then the creativity and the abduction interaction and being able to jump to an adjacent possible from what has been induced.

That's exciting.

Um, Arun?

And then we kind of have our last, uh, thoughts here.

Uh, I thought that all of those, uh, directions of future research sound really interesting.

Uh, I had a very possibly silly question, uh, which is on when we're talking about learning about the structure, are we also learning about potential new actions that an agent could take?

And I think that ties into the AI safety part quite nicely of currently an active inference.

If you come up with your own generative model as the modeler, I can say, these are the actions available to my agent.

It can only do these things.

It has to pick one of those things and it can't do anything else.

But if we're going to learn online about possible actions, like new actions I could take suddenly now I've learned that I have the choice to drive car.

Whereas before I could only walk, then suddenly the range of impacts that I could have my, me as the agent can have on the world massively increases and is possibly not predicted by the modeler at the time they're creating their agent.

So I think that, so yeah, my question is sort of about what type of structure do we want to learn with these new models and does that include actions that an agent could take?

Yeah.

Um, I mean, really interesting question.

And, um, so, so I'll give you a couple of thoughts.

Um, there is a Toby Sinclair Smith, uh, which you've probably had, had here on the show.

He, yeah, he, he does category theory and, and in his latest formulation of, um, structure learning, active inference with category theory, he talks about blankets that change over time.

And so this would be this idea of adding, removing actions of observations.

Um, so I think that's interesting and, and he would probably have some, some really cool thoughts about that.

The other perspective is let's say you fix the observation space and the action space.

Um, and let's suppose that each, that I, as a person exists only in a, like a small part of my head and my observations and actions are the information that's coming in and out of this small part of the head.

Now, when you're a baby, you don't know that you can act on your body, right?

So you don't know that your foot is you and your arm is you, but you're gonna learn eventually that there is a very precise mapping from the information that's coming out of your action neuron to what your arm can do.

And so eventually, even though your arm is a latent state, you're gonna associate moving my arm with an action that you're taking because there is a very precise mapping from your neuron to what the arm does.

And I think this would be, um, this would, basically when you, when you think about it this way, there is maybe a lot of actions that we can lump into the latent states of the environment.

the, that we have direct, uh, very precise control over that. That's the way I'm thinking about things.

Yeah, that's, that's the babbling. It's interesting. I'm also just thinking of expert video of figure skating,

but then there, that doesn't mean it's safe for a non-expert or there could have been a body

position that was safely reached. But then if a different object in a different context is so

that there's so many interesting, um, fundamental issues and that is the sort of benchmark

and reproducibility question is how can we compare models with these radically different forms, possibly in the future, also radically different substrates solving these problems in different ways,

such that the ways that benchmarks are even conceptualized in terms of like CPU or RAM may only be a constrained way to think about how some of these embodied problems are solved.

So, um, Lance, thank you for this and all the authors for the work.

We're very excited to, uh, talk with Carl and others in 58.2 during the Applied Active Inference Symposium.

So, good luck with your works. See you next time.

Thank you so much.

Thanks, everyone.

Thank you.

Thank you.